

Poster Presenter Guide — GPU Monte Carlo Option Pricing (poster_v3)

Layout note: v3 has no roofline panel, no bottom stat band, and no kernel cards in the overview. Your "big numbers" all live in the CONCLUSIONS panel (right column, bottom) — end your tour there.

The 60-second pitch

"Monte Carlo is the standard way to price options, but its error shrinks only as $1/\sqrt{N}$ — so you need tens of millions of simulated paths, and a CPU takes seconds per price. Each path is independent, which makes this a textbook GPU problem. We built three CUDA kernels, each isolating one performance lesson. The **naive** kernel gives every path its own thread and atomicAdd — the profiler shows the SMs doing useful work only 2.8% of the time, because millions of atomics serialize in L2. The **shared** kernel fixes that with a warp-shuffle + shared-memory reduction: one atomic per block instead of two per path, and SM throughput jumps to 79%. The **antithetic** kernel then halves the variance for free by mirroring every random draw. The change that mattered most was the **grid-stride refactor**: a fixed 16K-thread grid striding over paths makes RNG state $O(\text{threads})$ instead of $O(N)$ — init gets 65× cheaper and the kernels flip from memory-bound to compute-bound. End result: **382× end-to-end over a single-core CPU at 10 million paths, 980× at 50 million**, every run validated against the closed-form Black–Scholes price."

Walking order (v3 layout)

Start top-left, finish bottom-right:

1. **PROJECT OVERVIEW** (top-left) — problem, ground truth 10.4506, honest end-to-end methodology.
2. **THE THREE KERNELS — WHERE THEY DIFFER** (top-middle diagram) — your best visual: three columns, the only difference is how results are summed. Point at "compute SM = 2.8% / 79% / 76%" at the bottom of each card.
3. **1 · mc_naive** (left) — "94% occupancy but 2.8% compute: occupancy isn't utilization."
4. **2 · mc_shared** (left) — "Atomics go from $\sim 2 \cdot N$ to ~ 128 ; the reduction-tree diagram below is the whole trick."
5. **3 · mc_antithetic** (left) — "Pair +Z with -Z, noise cancels: half the variance for one extra exp()."
6. **4 · GRID-STRIDE REFACTOR (KEY)** (middle) — "The table is before/after: RNG init 44.6 ms $\rightarrow$ 0.68 ms. Biggest single win."

7. **BENCHMARK — THE RUNS** (middle) — say "median of 5 runs, warm-up discarded" before anyone asks.
8. **RESULTS — END-TO-END SPEED-UP / THROUGHPUT** (middle) — bar chart: 10× naive vs 348×/382× optimized at 10^7 ; line chart: naive flatlines at ≈ 415 M paths/s (atomic bus), optimized keep climbing.
9. **VALIDATION** (top-right) — convergence plot follows $1/\sqrt{N}$; table at $N=10^6$ shows all kernels statistically consistent.
10. **CONCLUSIONS** (bottom-right) — close with: 382×/980× end-to-end, $\approx 1650\times$ kernel-only, past the 200× course goal.

v3-specific caveats (know these before someone else spots them)

- **CONCLUSIONS says "all kernels validate within 1σ " but the VALIDATION table shows 1.4σ for naive and shared.** If challenged: "Fair catch — the precise statement is: every kernel is statistically consistent with the analytical price ($\leq 1.4\sigma$ at $N=10^6$, antithetic at 0.8σ); our automated acceptance gate is 4σ ." Do not argue the poster text.
- **The validation caption says the antithetic curve "sits below" the $0.5/\sqrt{N}$ reference** — at large N it doesn't, strictly. Say the dashed line is an $O(1/\sqrt{N})$ scaling guide, not a fitted bound.
- **The naive curve rises at 5×10^7 in the convergence plot.** Don't dodge it — it's a feature: FP32 atomic accumulation into one giant sum loses precision at that scale; the hierarchical reduction avoids it. Correctness argument, not just speed.
- **CONCLUSIONS says " $\approx 1650\times$ "** — the exact measured number is $1652\times$ (antithetic, kernel-only, 5×10^7).

Key numbers (memorize these five)

Number

382× / 980×

$\approx 1650\times$ (1652×)

2.8% → 79%

65×

10.4506

These five are the whole story: two say *how fast* (382×/980×, 1650×), two say *why* (2.8→79%, 65×), one says *against what truth* (10.4506).

How to explain each number

382× / 980× (end-to-end). Same job — simulate N paths and produce a price — timed wall-to-wall on both sides. One CPU core does ~ 30 M paths/s → ≈ 360 ms at 10^7 ; the GPU (mc_antithetic, RNG init + kernel + readback included) ≈ 0.9 ms → 382×. At 5×10^7 : ≈ 1.6 s vs ≈ 1.6 ms → 980×. Two numbers because speedup grows with N as GPU fixed costs amortize — quoting both is the honest version.

≈1650× (kernel-only; measured value 1652×). Same comparison but simulation kernel time only (init excluded). Read it together with 980×: "one price from cold start = 980×; many prices back-to-back, init paid once → approaches 1652×."

2.8% → 79% (SM throughput). Nsight's measure of how much of peak compute the SMs actually sustained. Naive: every path atomicAdds the same two global addresses → atomics serialize in L2 → cores idle ~97% → 2.8%, despite 94% occupancy (resident ≠ working). Shared: register → warp shuffle → shared-memory tree → one atomic per block (~128 total) → 79%. Same math, only the summation changed — 28× better utilization. This is the poster's core lesson.

65× (RNG init). One-thread-per-path needs N cuRAND states ($48 \text{ B} \times 10^6 \approx 46 \text{ MB}$); `init_rng` alone took 44.6 ms at $N=10^6$ — longer than the kernel itself. The fixed 16,384-thread grid-stride needs only 16,384 states → 0.68 ms = 65×. The side effect matters most: state traffic gone, kernels flip memory-bound → compute-bound.

10.4506 (ground truth). The Black-Scholes closed-form call price for $S_0=K=100$, $r=0.05$, $\sigma=0.2$, $T=1$. Every benchmark run is validated against it — it's what makes the speed claims trustworthy. Validation table at $N=10^6$: all kernels within $\leq 1.4\sigma$, antithetic at 0.8σ (σ = Monte Carlo standard-error units; see caveats above for the "within 1σ " wording).

Use them in this order: "We're 382× faster end-to-end — ≈1650× kernel-only — because we fixed the atomic bottleneck (2.8% → 79%) and made RNG init 65× cheaper; and every run validates against the analytical price 10.4506."

Demo line (if the live demo is running)

"Pick any N — price and runtime update instantly; the analytical value is printed next to it, so you can watch the error shrink as $1/\sqrt{N}$."

For every number and every claim on the poster, and full Q&A: see `poster_master_reference.md`.